

B9 B 系列

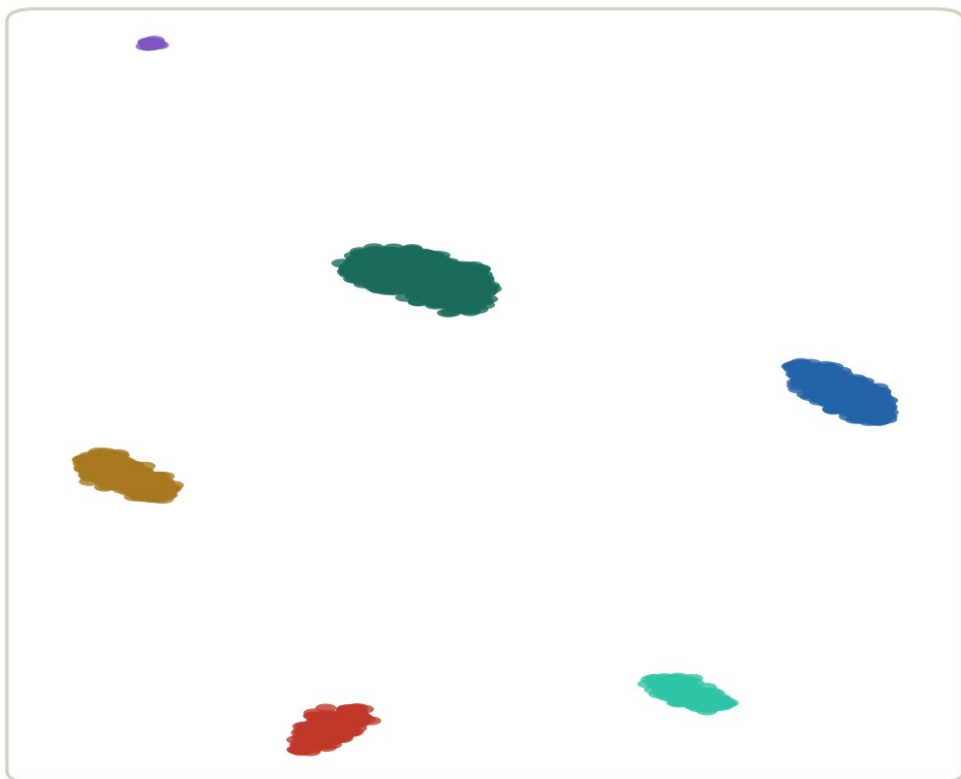
UMAP / t-SNE : 參數、隨機性與可重現性

設好 seed、講得出每個參數對圖形的影響——並且不再被 UMAP 騙。

約 30 分鐘 | 前置: B8 | 資料: PBMC 3k

同一份資料、同一行程式碼

你的電腦跑出來



同學的電腦跑出來



同一份資料
同一行程式碼



群的成員一顆都沒變——變的只有佈局。你的論文圖，別人重現得出來嗎？

模擬資料（UMAP 式嵌入實跑）

群的成員一模一樣，佈局卻不一樣。你的論文圖，三個月後重現得出來嗎？

本集的起點

圖的隨機性不處理， 你的論文就有一張沒人能重現的圖。

包括三個月後的你自己。這一集把隨機性抓出來、釘死、寫進方法段。

這一集你會學到

- 1 跑 RunUMAP / RunTSNE，並用 seed 的三個層級把結果釘死
- 2 用參數掃描說出 n.neighbors 與 min.dist 各控制什麼
- 3 複述 A5 的三大誤讀，並且做出一張出版級的 UMAP

01

UMAP 是顯示器，不是分析

先把它的地位擺對，參數焦慮少一半

分析都發生在 PC 空間

PC 空間 (10 維)

所有計算都發生在這裡

分群 (B8)

KNN 圖建在 PC 空間

距離與鄰居

誰跟誰像，這裡算

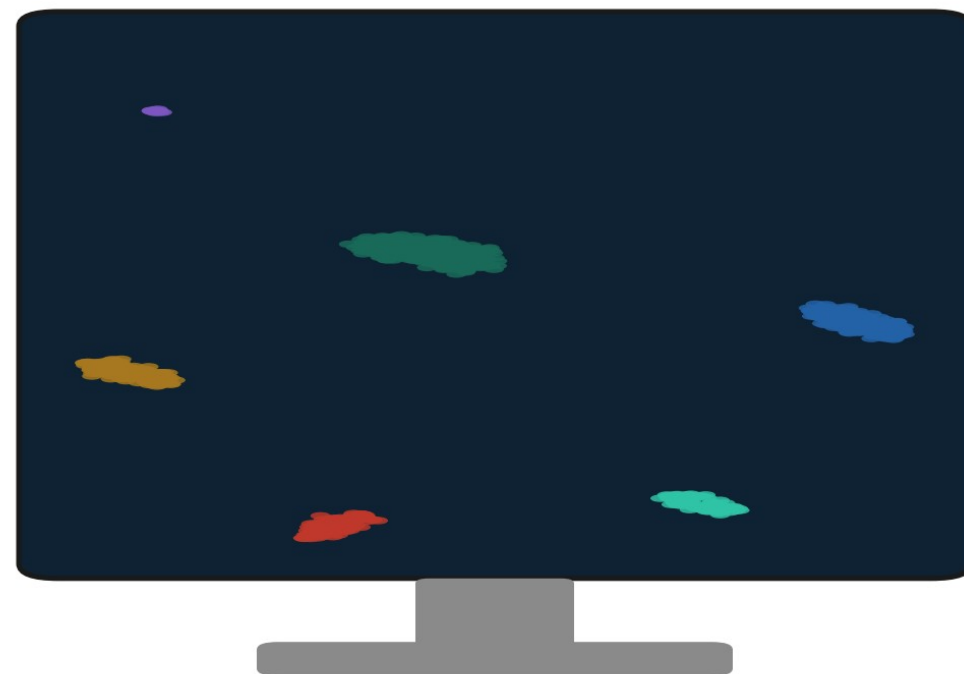
註釋 (B10) / DE (B12)

都不吃 UMAP 座標

RunUMAP

單行道：
只出不進

UMAP (2 維) = 顯示器



畫給眼睛看的投影；座標不進任何統計

分群、距離、註釋、DE 全部吃 PC 空間；UMAP 座標只負責給眼睛看

隨機性從哪裡來

① 初始化



每顆細胞在 2D 的「起點」
隨機撒或由譜方法給定

seed 管得到

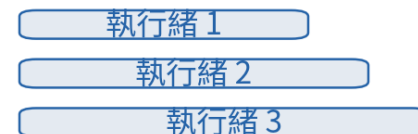
② 負採樣



SGD 每一步隨機抽
「非鄰居」來互相推開

seed 管得到

③ 平行化



多執行緒時浮點加法的
順序每次都不同

seed 管不到

前兩個來源用 seed 釘死；第三個要靠「單執行緒 + 記版本」——這就是等一下三個層級的由來

示意圖

- 初始化：每顆細胞在 2D 的起點
- 負採樣：SGD 每步隨機抽「非鄰居」來推開
- 平行化：多執行緒的加法順序，seed 管不到
- 前兩個用 seed 釘死；第三個靠單執行緒 + 記版本

本集核心觀念

UMAP 只是畫給眼睛看的投影。 座標不進任何統計。

所以參數與 seed 改變的只是「這張圖」，不是你的任何結論——前提是你沒拿座標去算東西。

02

RunUMAP 與 seed 的三個層級

先跑起來，再把隨機性一層一層釘死

RunUMAP：一行起跑

```
pbm c <- readRDS("output/pbm c_b08_clustered.rds") # B8 的 10 群  
  
pbmc <- RunUMAP(pbm c, dims = 1:10) # dims 承接 B7 的 nPC  
DimPlot(pbm c, reduction = "umap", label = TRUE)
```

Warning: The default method for RunUMAP has changed from calling Python UMAP via reticulate to the R-native UWOT using the cosine metric

這個 Warning 是資訊不是錯誤：v5 預設用 R 原生的 uwot 引擎，不需要裝 Python

RunUMAP 的預設值，逐個點名

dims = 1:10



不是預設，是你在
B7 決定的。

FindNeighbors 用
幾維，這裡就用幾維

**n.neighbors =
30、min.dist =
0.3**



兩個外觀旋鈕的預設，
下一節掃給你看

**seed.use =
42**



Seurat 已預設幫你
固定 seed——這是
你重跑會一樣的原因

重跑一次，驗證可重現

```
u1 <- Embeddings(pbm c, "um ap")
pbm c <- RunUMAP(pbm c, dims = 1:10) # 同參數再跑一次
u2 <- Embeddings(pbm c, "um ap")
all.equal(u1, u2)

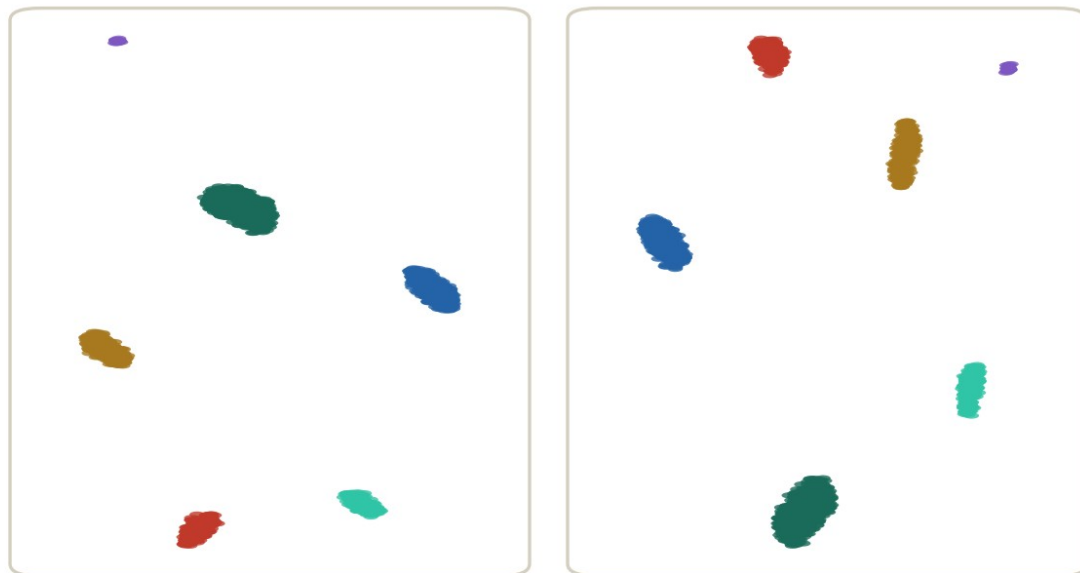
# 想看「另一種長相」：換 seed，其他都不動
pbm c <- RunUMAP(pbm c, dims = 1:10, seed.use = 123,
  reduction.name = "um ap_alt")
```

[1] TRUE

同機器、同版本、同參數 → 逐點一致。TRUE 的功臣是 `seed.use = 42`

seed 固定前後

沒固定 seed：兩次兩個樣



第一次跑

布局翻轉、旋轉、重排

第二次跑

模擬資料 (UMAP 式嵌入實跑)

固定 seed：跑幾次都同一張



第一次跑

逐點座標一模一樣

第二次跑

Seurat 的 RunUMAP 預設 seed.use = 42

左：seed 不固定，每次一個樣。右：seed 固定，跑幾次都同一張

寫進論文的方法段

**「UMAP : RunUMAP(dims = 1:10) ,
其餘含 seed.use = 42 皆為預設;
Seurat 與 uwot 版本鎖定於 renv.lock。」**

三個層級各就各位：全域 set.seed、函式 seed.use、實作層的版本與執行緒。缺一層，圖就可能漂。

03

兩顆旋鈕： `n.neighbors` 與 `min.dist`

不背定義，掃一遍看效果

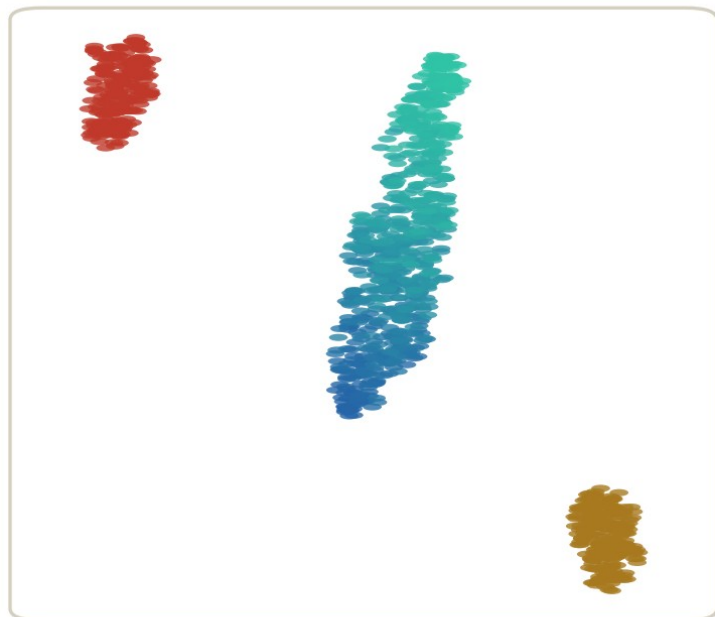
n.neighbors 掃描: 5 / 30 / 100

```
for (nn in c(5, 30, 100)) {  
  pbm c <- RunUMAP(pbm c, dim s = 1:10, n.neighbors = nn,  
    reduction.name = paste0("um ap_nn", nn),  
    reduction.key = paste0("UM APnn", nn, "_"),  
    verbose = FALSE)  
}  
p <- lapply(c(5, 30, 100), \(nn) Dim Plot(pbm c,  
  reduction = paste0("um ap_nn", nn)) + NoLegend())  
patchwork::wrap_plots(p, ncol = 3)
```

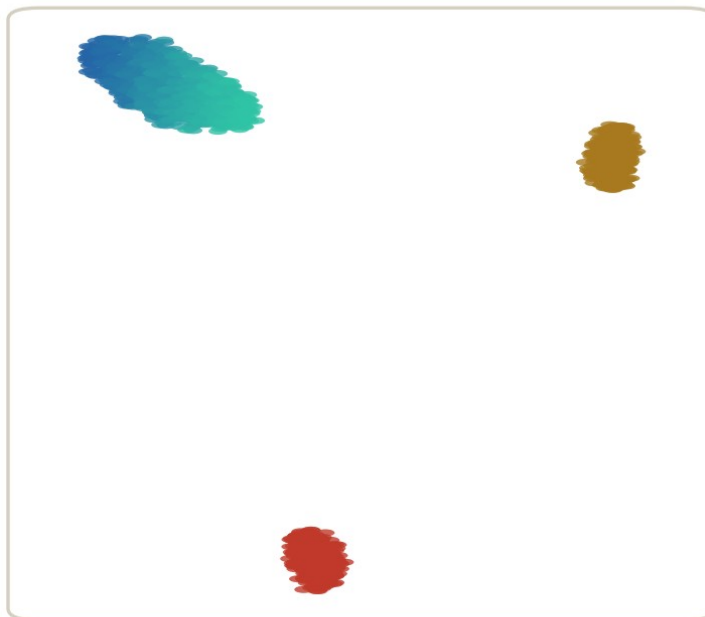
每個結果各存成一個 reduction，互不覆蓋——跟 B8 掃 resolution 同一個習慣

n.neighbors : 局部 vs 全域

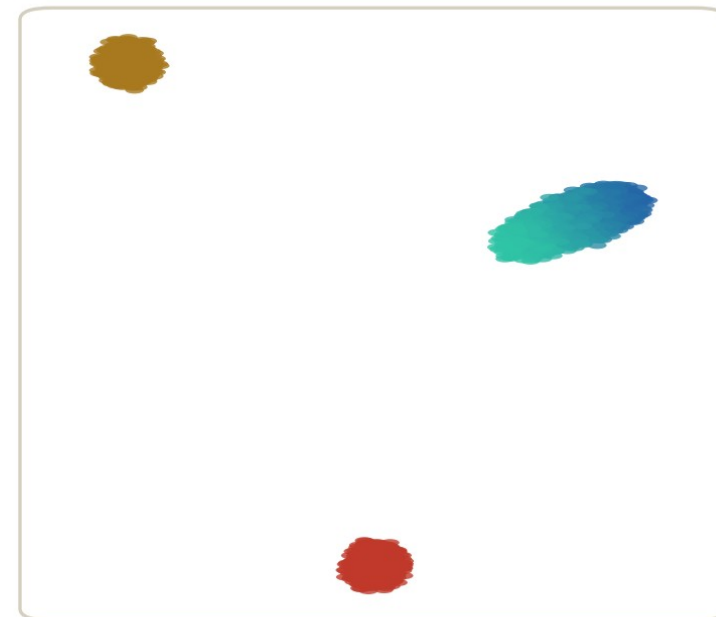
n.neighbors = 5



n.neighbors = 30 (預設)



n.neighbors = 100



只信 5 個鄰居：局部細節放大，帶被拉長

局部與全域的平衡點

看 100 個鄰居：全域優先，帶被壓扁

一條連續帶（藍→綠漸層）+ 兩個離散群——參數改的是視角，不是資料

模擬資料（UMAP 式嵌入實跑）

小：局部細節放大。大：全域結構優先，細節被抹平。預設 30 是折衷

min.dist 掃描: 0.01 / 0.3 / 0.8

```
for (m d in c(0.01, 0.3, 0.8)) {  
  tag <- gsub("\\.", "", as.character(m d)) # "001" "03" "08"  
  pbmc <- RunUMAP(pbmc, dims = 1:10, min.dist = m d,  
    reduction.name = paste0("umap_m d", tag),  
    reduction.key = paste0("UMAPm d", tag, "_"),  
    verbose = FALSE)  
}
```

min.dist: 低維空間裡「點跟點最近可以貼多近」。只影響外觀

min.dist：群內的鬆緊

min.dist = 0.01



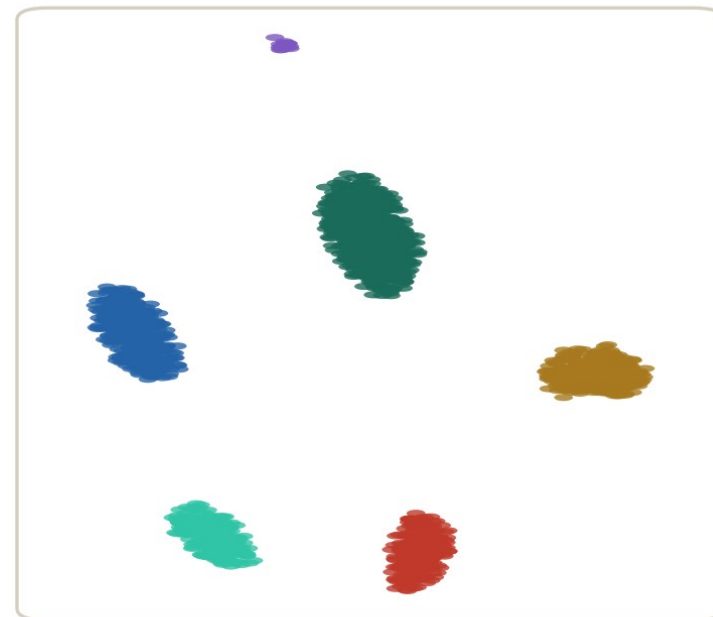
點可以貼死：群縮成硬塊，看起來「分超開」

min.dist = 0.3 (預設)



鬆緊適中

min.dist = 0.8



點被撐開：群鬆散、彼此靠近

min.dist 只管「群內的點可以靠多近」——它改變觀感，不改變任何分析結果

模擬資料 (UMAP 式嵌入實跑)

壓小：硬塊＋大片留白，看起來「分超開」。調大：鬆散、群彼此靠近

兩顆旋鈕，各管各的

記這張表，之後看別人的圖也用得上

n.neighbors

局部 vs 全域的取捨

- 改變 KNN 圖：這是「結構性」的旋鈕
- 小 → 局部細節；大 → 全域關係
- 對應 t-SNE 的 perplexity

min.dist

群內點的鬆緊

- 不動 KNN 圖：純外觀旋鈕
- 小 → 緊實硬塊；大 → 鬆散雲朵
- 留白多寡是它捏的，不是資料

這一節做了什麼

掃了兩個參數



六張圖存在物件裡，
隨時 DimPlot 對照

結論：預設夠用



PBMC 3k 用 30 /
0.3 就好；改參數要
有理由並寫下來

外觀 ≠ 證據



參數捏出來的分離不
能拿去說服任何人—
—踩雷區見

04

t-SNE：前輩還沒退休

一行對照跑完，知道何時還會遇到它

RunTSNE：一行對照

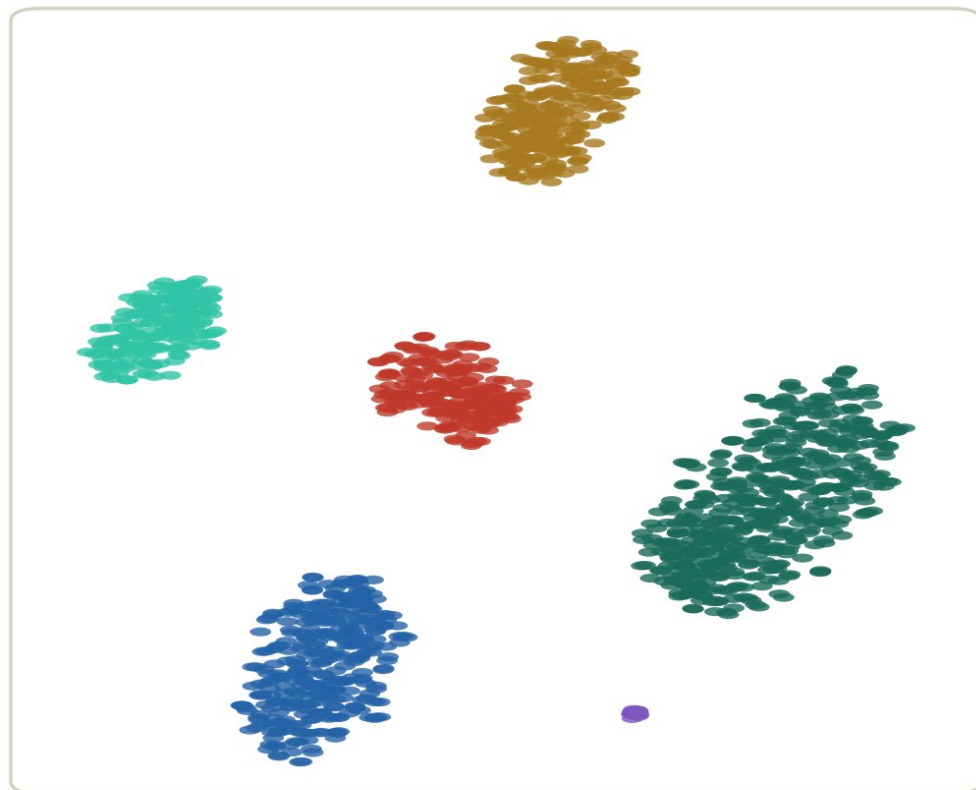
```
pbm c <- RunTSNE(pbm c, dim s = 1:10) # perplexity 預設 30
```

```
Dim Plot(pbm c, reduction = "tsne", label= TRUE) |  
Dim Plot(pbm c, reduction = "um ap", label= TRUE)
```

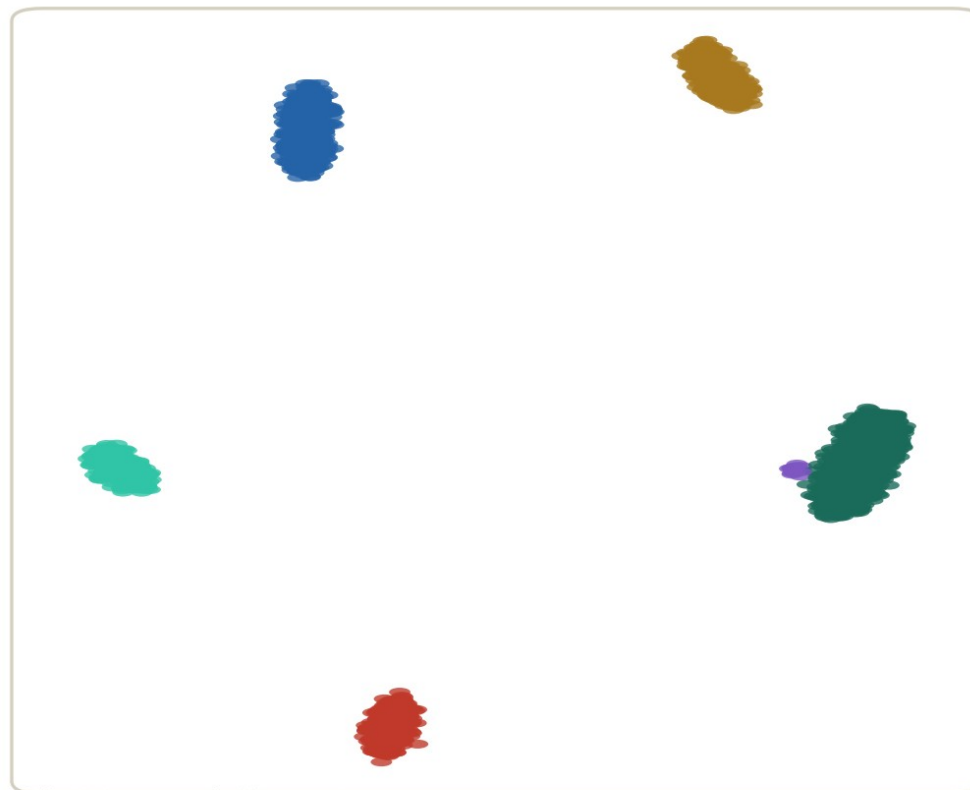
perplexity 之於 t-SNE，就是 n.neighbors 之於 UMAP：同一顆「局部範圍」旋鈕

t-SNE vs UMAP，同資料同顏色

t-SNE (perplexity = 30)



UMAP (n.neighbors = 30)



同一份資料
同一套顏色

perplexity 與 n.neighbors 是同一顆旋鈕：拿幾個鄰居定義「局部」

模擬資料 (t-SNE 為 sklearn 實跑)

兩張圖的群間距離都不可解讀

t-SNE 群間留白更平均——它的群間距離比 UMAP 更沒有意義

何時用哪個

預設用 UMAP；知道 t-SNE 什麼時候還會出場

UMAP

本系列的預設

- 快，大資料集也跑得動
- 全域結構保留得稍好
- 生態系主流：教學、圖庫、參考資料都用它

t-SNE

兩種場合還會遇到

- 重現或對照 2018 年以前的老論文
- 小資料集看局部結構，效果不輸 UMAP
- 群間距離連「參考」都不要參考

05

出版級的 DimPlot

讓審稿人一眼讀懂，也讓色盲讀者讀得懂

label、repel、色盲友善配色

```
cb10 <- c("#0072B2", "#E69F00", "#009E73", "#CC79A7",  
          "#56B4E9", "#D55E00", "#F0E442", "#999999",  
          "#882255", "#44AA99") # Okabe-Ito 延伸 10 色
```

```
p.pub <- DimPlot(pbm.c, reduction = "umap",  
                label = TRUE, repel = TRUE, label.size = 4,  
                cols = cb10, pt.size = 0.3) +  
  NoLegend() + ggtitle(NULL)
```

label 直接壓在群旁，repel 避免互相蓋字；NoLegend 省掉來回跳的圖例

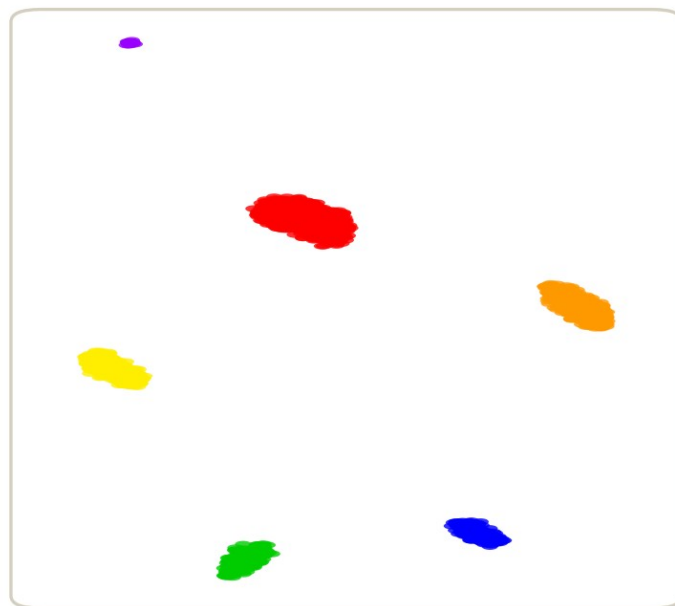
匯出：尺寸與字級一起決定

```
p.pub <- p.pub +  
  theme(axis.title = element_text(size = 9),  
        axis.text = element_text(size = 8))  
  
ggsave("output/B09_umap_final.pdf", p.pub,  
       width = 12, height = 10, units = "cm") # 半欄圖  
ggsave("output/B09_umap_final.png", p.pub,  
       width = 12, height = 10, units = "cm", dpi = 600)
```

先想圖在紙上多大，再回推字級：縮排後任何文字都不得小於 8 pt

災難版 vs 出版版

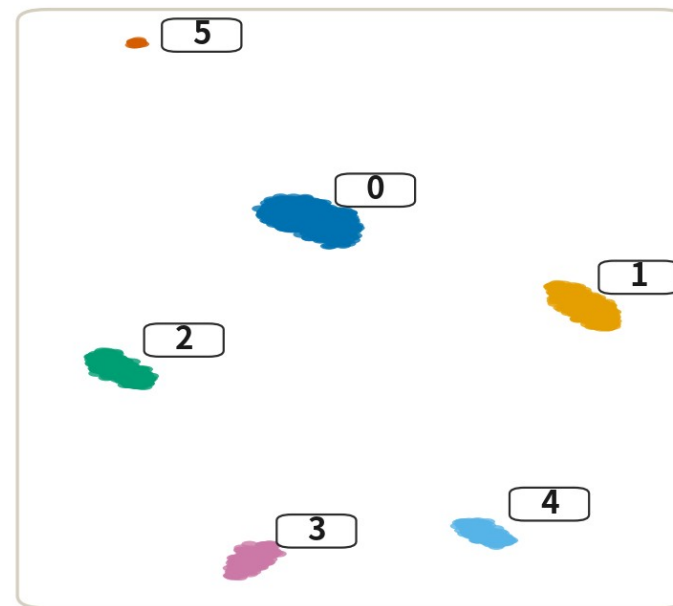
審稿人看到的：災難版



- cluster 0
- cluster 1
- cluster 2
- cluster 3
- cluster 4
- cluster 5

- ✗ 紅配綠：色盲讀者陣亡
- ✗ 編號在圖例，眼睛來回跳
- ✗ 6 pt 字級，印出來看不見

審稿人看到的：出版版



- ✓ Okabe-Ito 色盲友善配色
- ✓ 標籤壓在群旁 (repel)
- ✓ 匯出字級不小於 8 pt

同一份結果，兩種命運——出版級不是美學問題，是「讀者能不能無痛讀懂」的問題

圖說記得寫：Seurat 版本、dims、seed——圖跟參數是一組的

06

踩雷區

兩個雷，每一個都實際跑給你看

雷一：拿 UMAP 座標做下游統計

雷是什麼

用 UMAP 座標算群間距離、密度、軌跡，當成定量結果寫進論文。



為什麼會踩

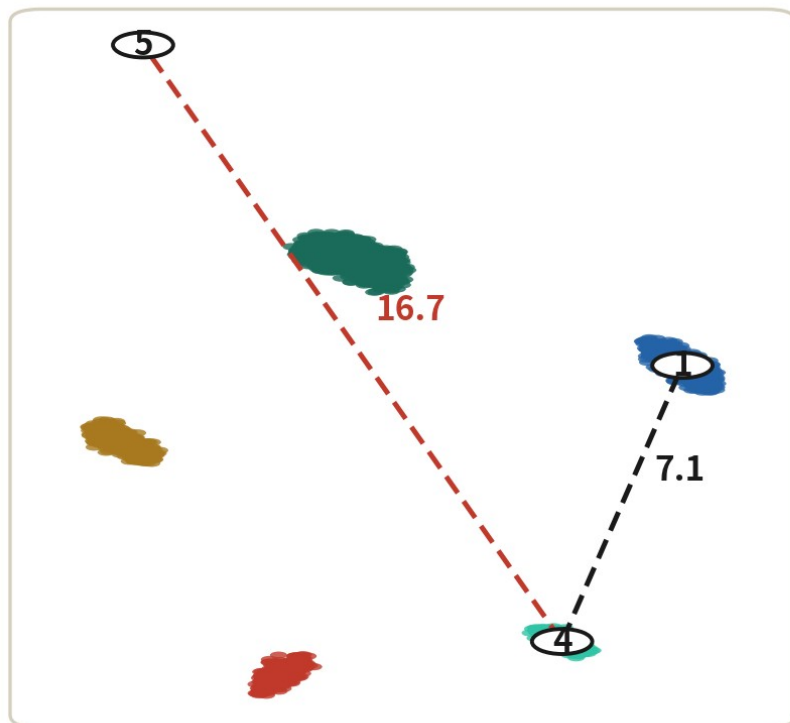
座標就在那裡，`dist()` 一行就算得出來，圖又長得很像「空間」。

踩了會怎樣

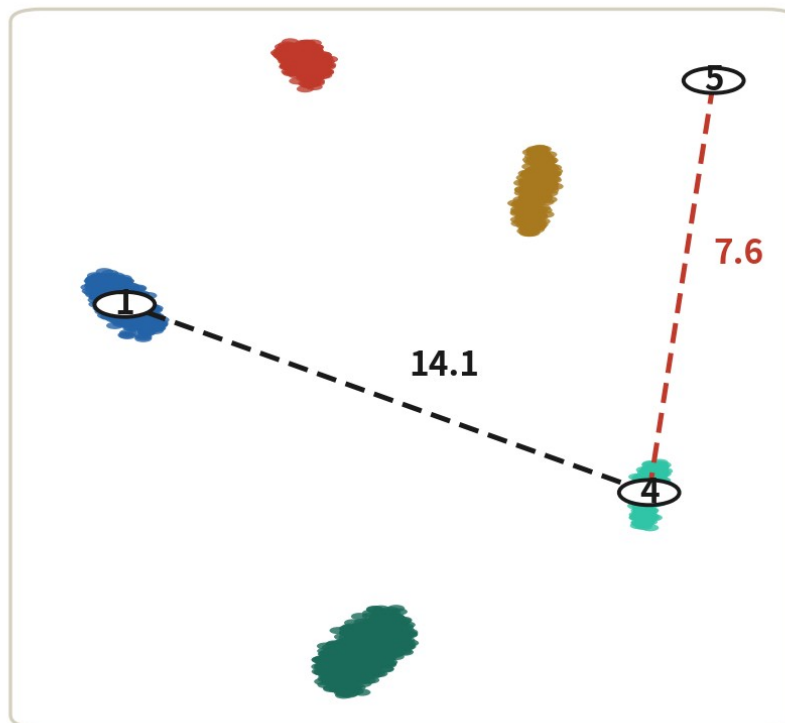
換一個 `seed`，「誰離誰近」整個翻轉。你的統計建立在流沙上。

實測：距離排序隨 seed 翻轉

seed A



seed B



同一份資料

seed A :
4 比較靠近 1

seed B :
4 比較靠近 5

15 組群間距離
排序相關性
 $\rho = -0.04$

「誰離誰近」隨 seed 翻轉——建在 UMAP 距離上的統計就是建在流沙上

模擬資料 (UMAP 式嵌入實跑)

seed A : 群 4 靠近群 1。 seed B : 群 4 靠近群 5。 15 組群間距離的排序相關性只剩 -0.04

雷二：調參數調到群分開為止

雷是什麼

兩群在圖上黏在一起，於是把 `min.dist`、`n.neighbors` 往小壓，直到「分乾淨」為止，拿去當兩群不同的證據。



為什麼會踩

跟 B8 的訂做群數同一個心理：圖長成我要的樣子，感覺就是證明。

踩了會怎樣

連 5% 過渡細胞的連續體都能被壓出「兩個界線分明的群」。圖上的留白是參數捏的，不是生物學。

實測：留白是可以訂做的

n.neighbors=50, min.dist=0.8



「一片連續體」

n.neighbors=5, min.dist=0.01



「兩個界線分明的群」

兩端族群+5% 過渡細胞——右邊只是把橋「藏」進參數裡 (repulsion.strength=2)

模擬資料 (UMAP 式嵌入實跑)

同一份資料，左邊「一片連續體」、右邊「兩群」。分不分開，證據永遠是 marker 與 PC 空間

這代表什麼

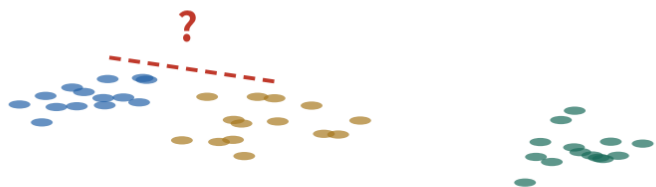
同一份資料
兩種結論

「圖上分得開」
不能當證據

證據要回
marker 與
PC 空間找

A5 三大誤讀，一頁複習

誤讀一：群間距離



✗ 「A 離 B 近，所以比較像」

✓ 距離要回 PC 空間算

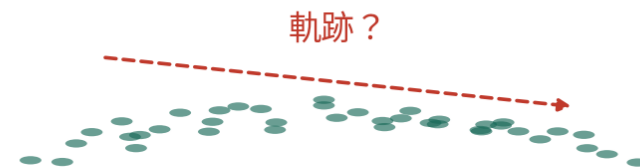
誤讀二：群面積



✗ 「這群好大，變異一定大」

✓ 面積是 min.dist 捏的

誤讀三：群形狀



✗ 「拉長條 = 分化軌跡」

✓ 形狀是最佳化的副產品

A5 講過的三句話，今天你已經有能力自己驗證：換個 seed、換個 min.dist，看它們怎麼變

示意圖

距離、面積、形狀——三個都不可解讀。今天你有工具自己驗證這三句話了

踩雷區小結

UMAP 上只有一件事可以信：
「這一群的成員是誰」。
其他都是佈景。

而成員名單根本不是 UMAP 算的——是 B8 在 PC 空間分好的。

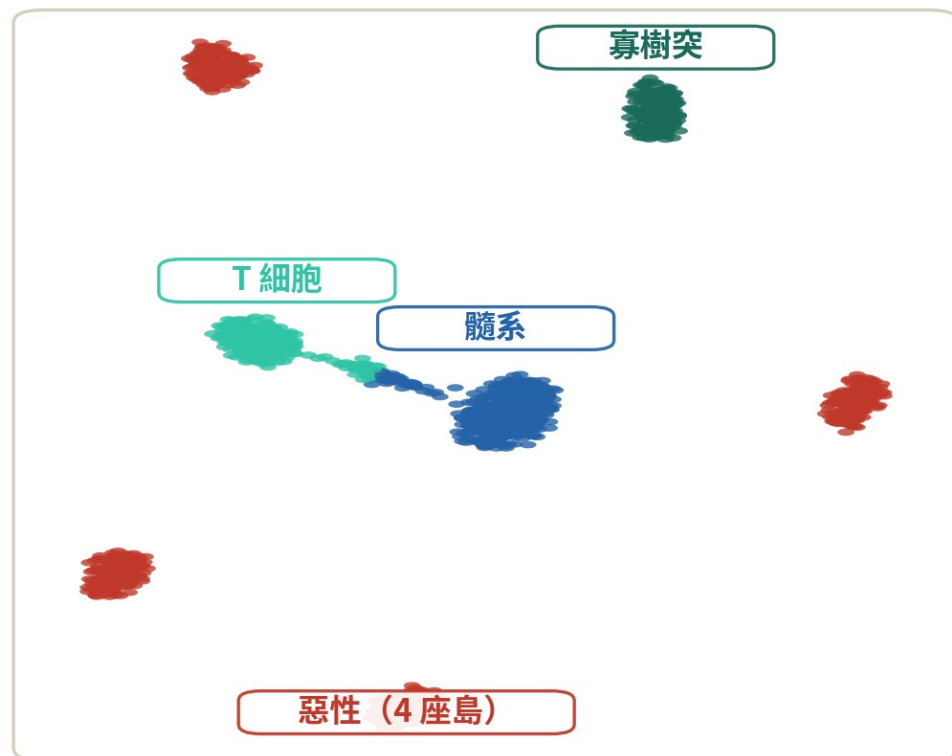
⑤ +

複雜樣本策略室

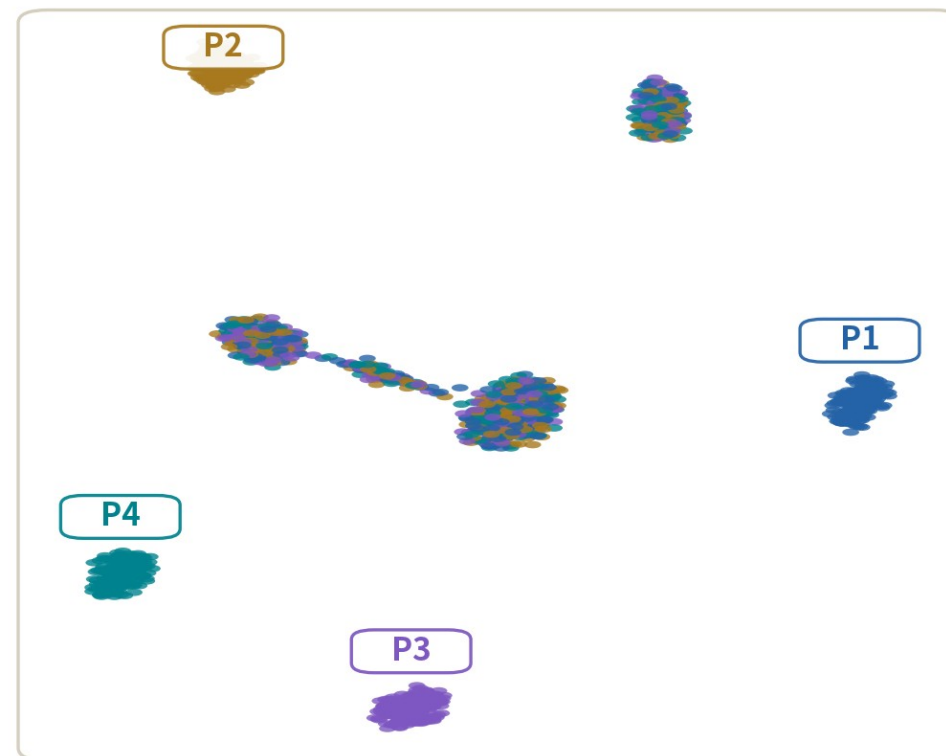
UMAP 你會讀了——現在想像圖上畫的不是 PBMC，是一顆腫瘤

先認四塊大陸： TME 相連，惡性成島

按細胞型別上色



同一張圖，按病人上色



TME 三塊：
跨病人混合

惡性一塊：
按病人成島

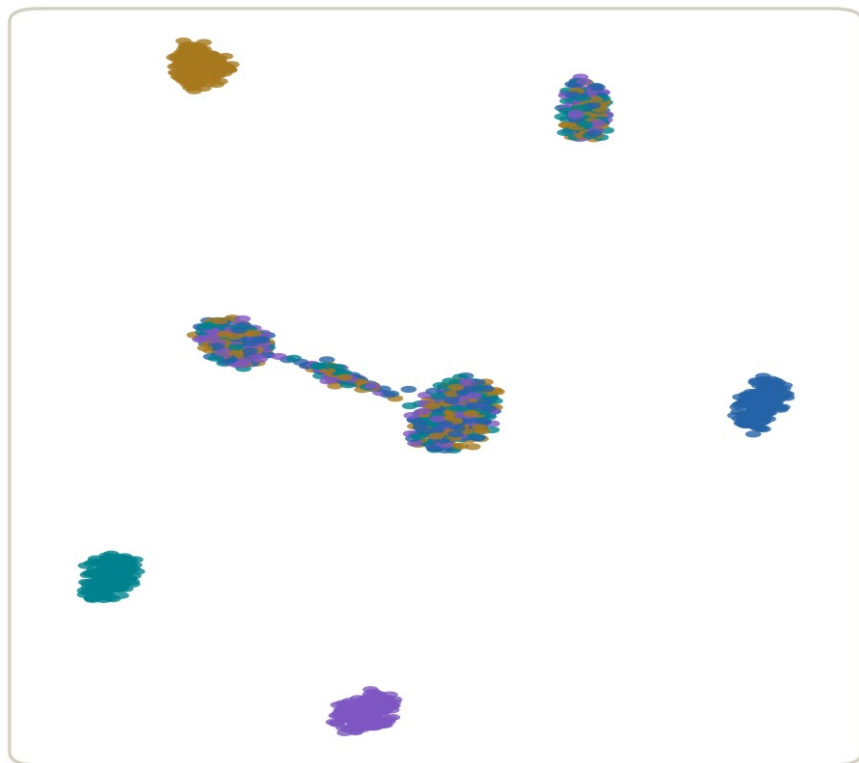
腫瘤 UMAP 第一眼：先認四塊大陸——TME 跨病人混合、彼此相連；惡性細胞一人一座島

模擬資料（UMAP 式嵌入實跑）

髓系、T 細胞、寡樹突跨病人混合；惡性細胞一人一座島——因為每人的 CNV 組合不同

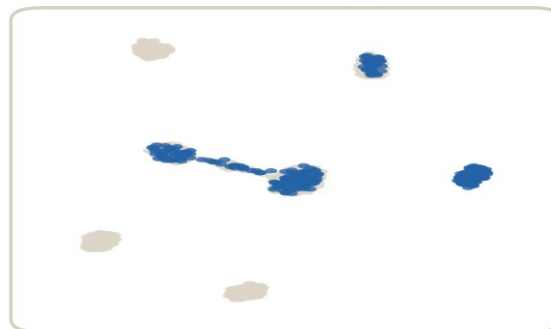
per-patient 並排：腫瘤的標準圖

全體：按病人上色



split.by = "patient"：每格一位病人

P1 • 343 顆



P2 • 347 顆



P3 • 343 顆



P4 • 357 顆



並排回答三個問題：哪座島是誰私有的？TME 每位病人都有貢獻嗎？誰的細胞少到不能單獨下結論？

模擬資料（UMAP 式嵌入實跑）

group.by 加 split.by 兩個參數——工具這一集就教過了，差別在你知不知道要畫這組圖

同樣的誤讀，在惡性大陸上加倍危險

距離、形狀在 PBMC 上已經不可信；到了腫瘤又疊上病人與整合兩層

PBMC

換個 seed 就翻盤

- 群間距離：排序相關性 ≈ 0
- 形狀：最佳化的副產品
- 但群的成員名單還是可信的

腫瘤

島距混進病人身分與整合強度

- 兩位病人的島靠得近 $\neq$ 亞型相似
- 惡性分四群 $\neq$ 四種亞型——那是四位病人；亞型看狀態分數
- 整合一開，島可以被拉成一團（B11）

本集 checklist



RunUMAP 的 dims 承接 B7 的 1:10，理由說得出口



seed 三個層級都處理了：set.seed、seed.use、單執行緒+記版本



n.neighbors 與 min.dist 掃過，知道各自控制什麼



分析全部在 PC 空間；UMAP 座標不進任何統計



A5 三大誤讀（距離／面積／形狀）能自己複述



出版圖：色盲友善、標籤上圖、字級夠大、參數寫進圖說

六格全勾，才進 B10

一句話總結

**分析在 PC 空間，圖在 UMAP；
seed 釘死，參數寫下來。**

做到這兩句，你的圖既誠實又可重現。

自測 1

分群、算細胞間距離、做 DE——這些分析該用哪個空間？

- A UMAP 的二維座標
- B PC 空間（本系列為前 10 個 PC）
- C 原始基因表現矩陣，不降維
- D t-SNE 座標，比 UMAP 可靠

答 B。分群的 KNN 圖、距離、下游統計全部建立在 PC 空間；UMAP 與 t-SNE 都只是畫給眼睛看的投影，座標不進任何計算。

自測 2

兩張 UMAP：一張群緊實、留白大（`min.dist=0.01`），一張鬆散、群靠近（`min.dist=0.8`）。能說前者「分群品質較好」嗎？

- A 能，留白大代表群分得開
- B 能，但要再看 `n.neighbors`
- C 不能，`min.dist` 只改外觀，分群結果一顆細胞都沒變
- D 不能比較，因為 `seed` 不同

答 C。分群在 B8 就定案了，UMAP 只負責畫。`min.dist` 捏的是點跟點可以貼多近——留白是參數的作品，不是品質指標。

自測 3

審稿人要求「圖可重現」。哪一組做法才算齊全？

- A 腳本開頭 `set.seed(1234)`，這樣就夠
- B `seed.use` 固定 + 版本記錄（`renv/sessionInfo`） + 預設單執行緒的 `uwot` + 可重跑的腳本
- C 多跑幾次，挑最好看的存檔備用
- D 改用 `t-SNE`，它沒有隨機性

答 B。全域 `set.seed` 管不到 `RunUMAP` 內部；要函式層的 `seed.use`、實作層的版本與執行緒一起釘，配上能從頭跑到尾的腳本。A 只蓋了一層，C 是掩耳盜鈴，D 更錯——`t-SNE` 一樣隨機。

下集預告

B10：細胞註釋—— marker、SingleR、Azimuth

圖上有十個群，但它們現在只叫 0 到 9。哪群是 T 細胞？哪群是單核球？

下一集：手動與自動兩條註釋路線——以及兩邊吵架的時候，聽誰的。